

보안 강화를 위한 API 키 대체 인증 방식

정적인 장기(Long-lived) API 키는 소스 코드 유출 등의 위험이 있으므로, Google은 환경 및 사용 사례에 따라 더 강력한 인증 방식을 채택할 것을 권장합니다.

대안 1: 서버 간 통신을 위한 애플리케이션 기본 사용자 인증 정보 (ADC)

서버 투 서버(예: Cloud Functions에서 Gemini API 호출) 인증 시 가장 권장되고 안전한 방법입니다.

- **원리:** 클라이언트 라이브러리가 애플리케이션 환경을 기반으로 사용자 인증 정보를 자동으로 찾습니다.
- **장점:** 코드 내에 자격 증명 파일이나 키를 저장할 필요가 없습니다. Google이 자동으로 관리하고 순환(Rotate)시키는 수명이 짧은 액세스 토큰을 사용하므로 보안 위험이 크게 감소합니다.
- **적용:** 각 서비스마다 전용 서비스 계정(Service Account)을 생성하고 최소 권한(예: roles/aiplatform.user)을 부여하여 배포 시 연결합니다.

참조 문서:

- [Google Cloud - 애플리케이션 기본 사용자 인증 정보\(ADC\) 설정](#)
- [Google Cloud - 서비스 계정 개요](#)

대안 2: 백엔드 환경을 위한 Google Cloud Secret Manager

API 키를 불가피하게 백엔드에서 사용해야 할 경우 하드코딩을 방지하는 모범 사례입니다.

- **원리:** Secret Manager에 키를 안전하게 저장하고 런타임에 프로그래밍 방식으로 액세스합니다.
- **제어:** 키를 필요로 하는 특정 서비스 계정에만 IAM 역할(roles/secretmanager.secretAccessor)을 부여하여 최소 권한을 강제합니다.

참조 문서: [Google Cloud Secret Manager 개요](#)

대안 3: 최종 사용자 인증을 위한 OAuth 2.0 프레임워크

애플리케이션이 최종 사용자를 대신하여 Google Cloud 서비스에 액세스해야 할 때 적합한 방법입니다.

- **원리:** 사용자의 명시적 동의(Consent)를 얻는 권한 부여 프로토콜입니다.
- **흐름(Authorization Code Flow):** 앱이 사용자를 Google 권한 부여 서버로 리디렉션하고, 동의를 얻은 후 부여받은 임시 코드를 통해 수명이 짧은 액세스 토큰(Access Token)과 장기 리프레시 토큰(Refresh Token)으로 교환합니다. 요청 헤더(Authorization: Bearer <token>)에 액세스 토큰을 포함하여 사용합니다.

참조 문서: [Google Identity - OAuth 2.0을 사용하여 Google API 액세스](#)